Istio Service Mesh Getting Started Experiment Report

Experiment Overview

This experiment aims to deploy and evaluate Istio service mesh capabilities on a Windows environment using a local Kubernetes cluster. The experiment follows the official Istio getting started documentation and covers the complete workflow from installation to observability.

Environment Setup

The experiment was conducted on a Windows machine with PowerShell as the primary command line interface. The Kubernetes cluster was provisioned locally. The first step involved downloading the Istio release package for Windows. Two versions were available with nearly identical content: istio-1.29.2-win-amd64.zip and istio-1.29.2-win.zip. The sha256 files served as integrity verification checksums. After extraction, the actual directory structure was C:\istio-1.29.2-win\istio-1.29.2, containing the bin directory with istioctl.exe and the samples directory with demo applications.

Installation Process

The istioctl binary was added to the system PATH. On Windows, the temporary PATH modification using $env:PATH only persists within the current PowerShell session, which means any new terminal window requires the path to be set again. For permanent configuration, the system environment variable

needs to be updated through the Windows settings or via the [Environment]::SetEnvironmentVariable method.

Istio was installed using the demo profile without default gateways. The command used was istioctl install with the demo-profile-no-gateways.yaml configuration file. This profile is optimized for testing and evaluation purposes. After installation, the default namespace was labeled with istio-injection=enabled to enable automatic Envoy sidecar proxy injection for all subsequently deployed workloads.

Gateway API Configuration

Before deploying applications, the Kubernetes Gateway API Custom Resource Definitions were installed. These CRDs are not included by default in most Kubernetes distributions. The installation used kubectl kustomize to fetch the v1.4.0 version of the gateway API configuration directly from the upstream repository.

Application Deployment

The Bookinfo sample application was deployed. This application consists of four microservices: productpage, details, reviews, and ratings. Notably, the reviews service has three versions (v1, v2, v3) to demonstrate traffic splitting and canary deployment capabilities. Upon deployment, each pod showed a READY status of 2 out of 2 containers, confirming that the Istio sidecar proxy

was successfully injected alongside the application container. The application was verified internally by executing a curl command from the ratings pod to the productpage service, which returned the expected HTML title response.

Traffic Exposure

To expose the application externally, a Kubernetes Gateway and HTTPRoute were created using the provided bookinfo-gateway.yaml manifest. Since the experiment environment did not have a cloud load balancer, the gateway service type was changed to ClusterIP via annotation. This allowed local access through port forwarding rather than requiring an external IP address.

Access Verification

The application was accessed through kubectl port-forward, mapping port 8080 on the local machine to port 80 of the gateway service. The Bookinfo application was viewable at localhost:8080/productpage. Refreshing the page multiple times demonstrated the load balancing behavior across the three versions of the reviews service, with visible differences in the star ratings display: no stars for v1, black stars for v2, and red stars for v3.

Observability Setup

Kiali was installed as the primary observability dashboard for the service mesh. Initially, Kiali was deployed without its dependency on Prometheus, which resulted in an error when attempting to load the traffic graph. The error

message indicated a failure to query Prometheus at prometheus.istio-system:9090. After installing the Prometheus addon, Kiali was able to retrieve metrics data successfully.

Traffic Generation and Graph Visualization

Initially, the Kiali graph appeared empty because no measurable traffic had been recorded in the default namespace within the selected time window. To populate the graph, traffic was generated by repeatedly accessing the productpage endpoint. Once sufficient requests were made, the Kiali dashboard displayed the complete service topology, showing the relationships between productpage, details, reviews, and ratings services, along with traffic distribution percentages and success rates.

Challenges and Reflections

Throughout this experiment, several issues were encountered that provided valuable learning opportunities. The first challenge involved the Windows file system path. The extracted Istio directory contained a nested folder structure, and the initial PATH configuration pointed to an incorrect location, causing the system to fail to recognize the istioctl command. This was resolved by carefully verifying the actual directory structure using Get-ChildItem.

The second issue related to PowerShell session management. Environment variables set via $env:PATH are session-scoped and do not persist across new

terminal windows. This caused repeated command not found errors whenever a new PowerShell window was opened. The lesson learned is that for Windows users, either the system PATH must be permanently configured, or the full path to the binary must be used in every new session.

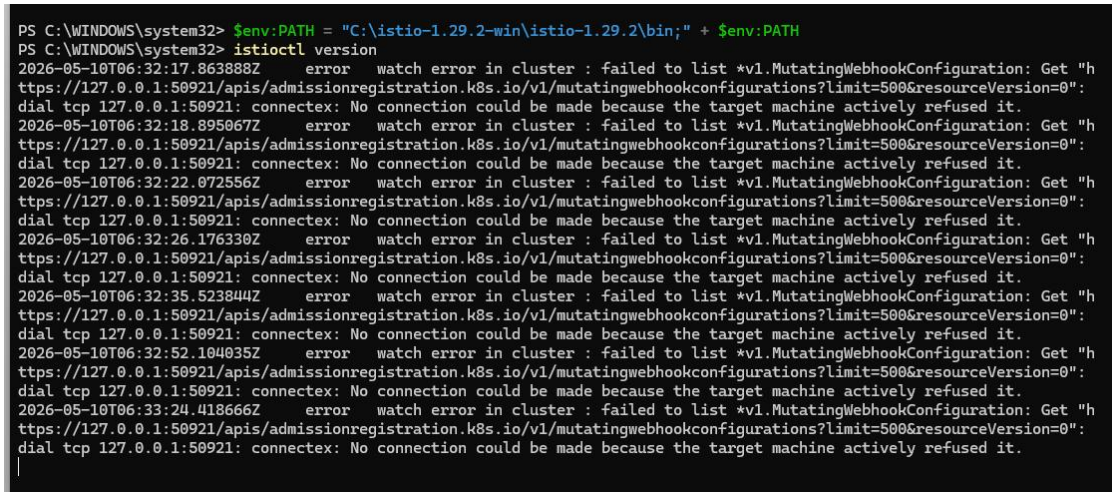
The third challenge was the Kubernetes cluster connectivity. Initially, istioctl could not connect to the API server because the local cluster was not running. This highlighted the importance of ensuring the cluster is active and kubectl is properly configured before attempting any Istio operations.

The fourth issue occurred during the Kiali setup. Installing Kiali alone without Prometheus resulted in graph loading failures. This demonstrated that Istio's observability stack consists of multiple integrated components, and Kiali depends on Prometheus as its metrics backend. Understanding these dependencies is crucial for troubleshooting observability issues in production environments.

The fifth and most recurring issue involved the kubectl port-forward command. Port forwarding is a blocking operation that occupies the terminal session. Attempting to execute additional commands in the same window either fails or prevents the port forward from functioning correctly. The correct approach is to dedicate a separate terminal window solely for port forwarding and use other windows for application interaction and traffic generation.

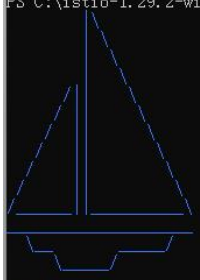
Overall, this experiment successfully demonstrated the core capabilities of Istio including sidecar injection, gateway traffic management, service-to-service communication, and observability through Kiali. The challenges encountered primarily stemmed from environment configuration and understanding of command behavior rather than Istio itself. These experiences reinforce the importance of careful path management, session awareness, and understanding component dependencies when working with service mesh technologies on Windows environments.

Screenshot



```
PS C:\WINDOWS\system32> $env:PATH = "C:\istio-1.29.2-win\istio-1.29.2\bin;" + $env:PATH
>> istioctl version
Istio is not present in the cluster: no running Istio pods in namespace "istio-system"
client version: 1.29.2
PS C:\WINDOWS\system32>
```

```
PS C:\WINDOWS\system32> $env:PATH = "C:\istio-1.29.2-win\istio-1.29.2\bin;" + $env:PATH
>> istioctl version
Istio is not present in the cluster: no running Istio pods in namespace "istio-system"
client version: 1.29.2
PS C:\WINDOWS\system32> cd C:\istio-1.29.2-win\istio-1.29.2
>>
PS C:\istio-1.29.2-win\istio-1.29.2> istioctl install -f samples/bookinfo/demo-profile-no-gateways.yaml -y
```



```
✔Istio core installed ⛵

✔Istiod installed 🧠
✔Installation complete
PS C:\istio-1.29.2-win\istio-1.29.2>
```

```
PS C:\istio-1.29.2-win\istio-1.29.2> kubectl label namespace default istio-injection=enabled
namespace/default labeled
PS C:\istio-1.29.2-win\istio-1.29.2> kubectl get crd gateways.gateway.networking.k8s.io 2>$null
>> if ($LASTEXITCODE -ne 0) {
>>   kubectl kustomize "github.com/kubernetes-sigs/gateway-api/config/crd?ref=v1.4.0" | kubectl apply -f -
>> }
customresourcedefinition.apiextensions.k8s.io/backendtlspolicies.gateway.networking.k8s.io created
customresourcedefinition.apiextensions.k8s.io/gatewayclasses.gateway.networking.k8s.io created
customresourcedefinition.apiextensions.k8s.io/gateways.gateway.networking.k8s.io created
customresourcedefinition.apiextensions.k8s.io/grpcroutes.gateway.networking.k8s.io created
customresourcedefinition.apiextensions.k8s.io/httproutes.gateway.networking.k8s.io created
customresourcedefinition.apiextensions.k8s.io/referencegrants.gateway.networking.k8s.io created
PS C:\istio-1.29.2-win\istio-1.29.2>
```

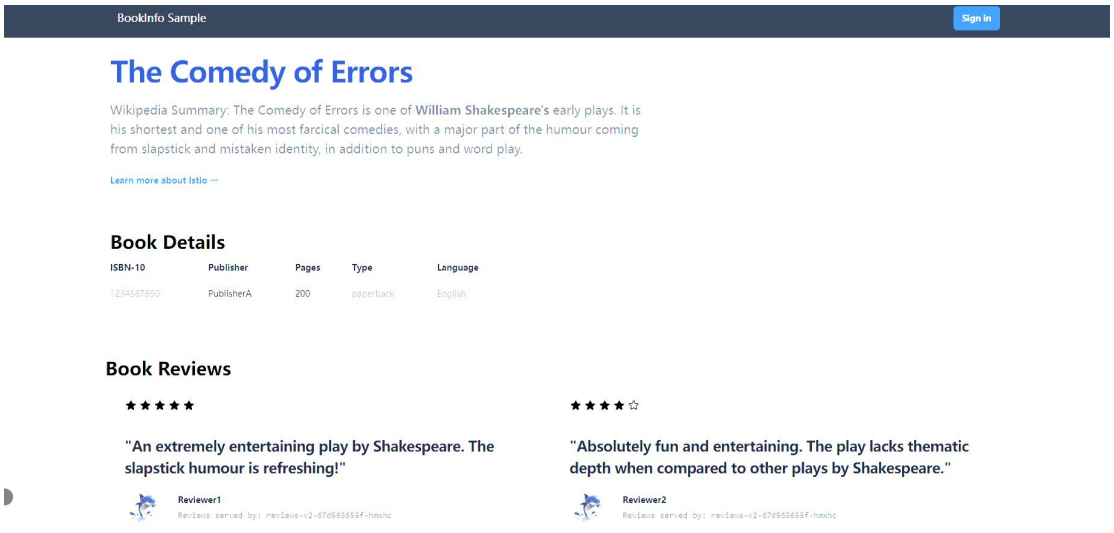
```
PS C:\istio-1.29.2-win\istio-1.29.2> kubectl get crd gateways.gateway.networking.k8s.io 2>$null
>> if ($LASTEXITCODE -ne 0) {
>>   kubectl kustomize "github.com/kubernetes-sigs/gateway-api/config/crd?ref=v1.4.0" | kubectl apply -f -
>> }
customresourcedefinition.apiextensions.k8s.io/backendtlspolicies.gateway.networking.k8s.io created
customresourcedefinition.apiextensions.k8s.io/gatewayclasses.gateway.networking.k8s.io created
customresourcedefinition.apiextensions.k8s.io/gateways.gateway.networking.k8s.io created
customresourcedefinition.apiextensions.k8s.io/grpcroutes.gateway.networking.k8s.io created
customresourcedefinition.apiextensions.k8s.io/httproutes.gateway.networking.k8s.io created
customresourcedefinition.apiextensions.k8s.io/referencegrants.gateway.networking.k8s.io created
PS C:\istio-1.29.2-win\istio-1.29.2> kubectl apply -f samples/bookinfo/platform/kube/bookinfo.yaml
service/details created
serviceaccount/bookinfo-details created
deployment.apps/details-v1 created
service/ratings created
serviceaccount/bookinfo-ratings created
deployment.apps/ratings-v1 created
service/reviews created
serviceaccount/bookinfo-reviews created
deployment.apps/reviews-v1 created
deployment.apps/reviews-v2 created
deployment.apps/reviews-v3 created
service/productpage created
serviceaccount/bookinfo-productpage created
deployment.apps/productpage-v1 created
PS C:\istio-1.29.2-win\istio-1.29.2>
```

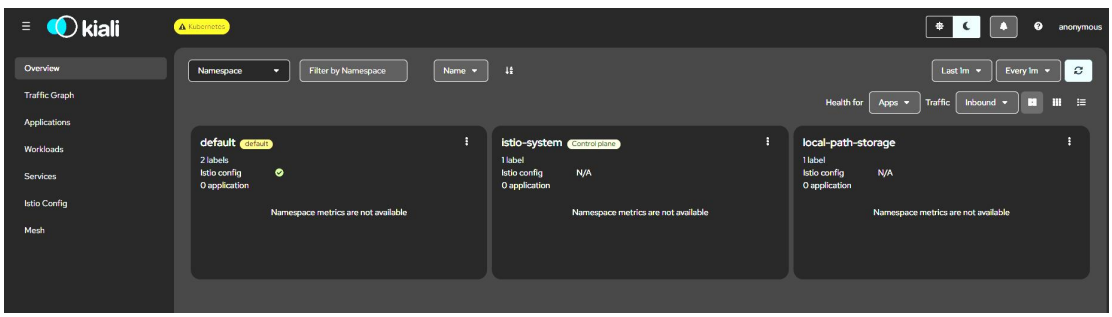
```
PS C:\istio-1.29.2-win\istio-1.29.2> kubectl get pods
>> kubectl get services
NAME                              READY   STATUS            RESTARTS   AGE
details-v1-77d6bd5675-rkrcr       2/2     Running           0          108s
productpage-v1-bb87ff47b-rdsv6    1/2     PodInitializing   0          108s
ratings-v1-8589f64b4c-7g88m       2/2     Running           0          108s
reviews-v1-8cf7b9cc5-cxz6g        2/2     Running           0          108s
reviews-v2-67d565655f-hmxhc       1/2     PodInitializing   0          108s
reviews-v3-d587fc9d7-tkdsm        1/2     PodInitializing   0          108s
NAME          TYPE        CLUSTER-IP      EXTERNAL-IP   PORT(S)    AGE
details       ClusterIP   10.96.214.67    <none>        9080/TCP   108s
kubernetes    ClusterIP   10.96.0.1       <none>        443/TCP    7m32s
productpage   ClusterIP   10.96.9.3       <none>        9080/TCP   108s
ratings       ClusterIP   10.96.8.237     <none>        9080/TCP   108s
reviews       ClusterIP   10.96.4.14      <none>        9080/TCP   108s
PS C:\istio-1.29.2-win\istio-1.29.2>
PS C:\istio-1.29.2-win\istio-1.29.2>
```

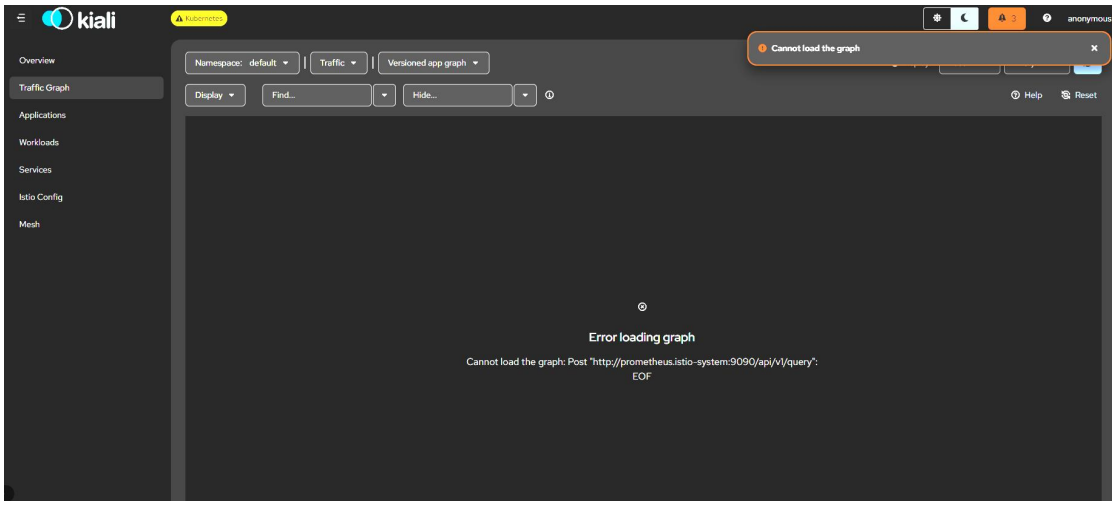
```
PS C:\istio-1.29.2-win\istio-1.29.2> $ratingsPod = kubectl get pod -l app=ratings -o jsonpath='{.items[0].metadata.name}'
>> kubectl exec $ratingsPod -c ratings -- curl -sS productpage:9080/productpage | findstr "<title>"
<title>Simple Bookstore App</title>
```

```
PS C:\istio-1.29.2-win\istio-1.29.2> kubectl apply -f samples/bookinfo/gateway-api/bookinfo-gateway.yaml
>> kubectl annotate gateway bookinfo-gateway networking.istio.io/service-type=ClusterIP --namespace=default
>> kubectl get gateway
gateway.gateway.networking.k8s.io/bookinfo-gateway created
httproute.gateway.networking.k8s.io/bookinfo created
gateway.gateway.networking.k8s.io/bookinfo-gateway annotated
NAME               CLASS   ADDRESS   PROGRAMMED   AGE
bookinfo-gateway   istio             False        0s
PS C:\istio-1.29.2-win\istio-1.29.2>
```

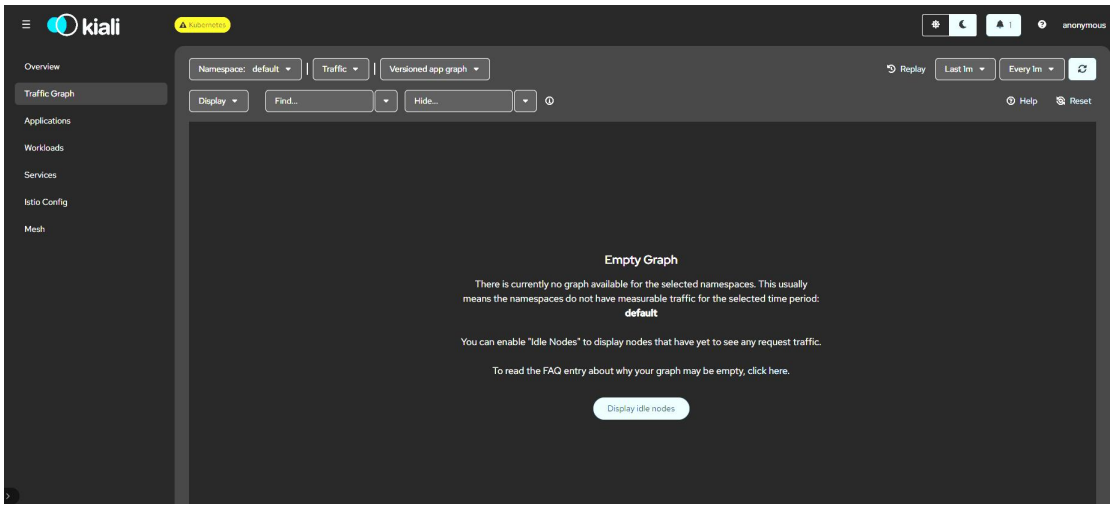
```
PS C:\istio-1.29.2-win\istio-1.29.2> kubectl apply -f samples/bookinfo/gateway-api/bookinfo-gateway.yaml
>> kubectl annotate gateway bookinfo-gateway networking.istio.io/service-type=ClusterIP --namespace=default
>> kubectl get gateway
gateway.gateway.networking.k8s.io/bookinfo-gateway created
httproute.gateway.networking.k8s.io/bookinfo created
gateway.gateway.networking.k8s.io/bookinfo-gateway annotated
NAME               CLASS   ADDRESS   PROGRAMMED   AGE
bookinfo-gateway   istio             False        0s
PS C:\istio-1.29.2-win\istio-1.29.2>
```








```
PS C:\WINDOWS\system32> cd C:\istio-1.29.2-win\istio-1.29.2
PS C:\istio-1.29.2-win\istio-1.29.2> kubectl apply -f samples/addons/prometheus.yaml
serviceaccount/prometheus created
configmap/prometheus created
clusterrole.rbac.authorization.k8s.io/prometheus created
clusterrolebinding.rbac.authorization.k8s.io/prometheus created
service/prometheus created
deployment.apps/prometheus created
PS C:\istio-1.29.2-win\istio-1.29.2> kubectl rollout status deployment/prometheus -n istio-system
Waiting for deployment "prometheus" rollout to finish: 0 of 1 updated replicas are available...
deployment "prometheus" successfully rolled out
PS C:\istio-1.29.2-win\istio-1.29.2> kubectl apply -f samples/addons/grafana.yaml
PS C:\istio-1.29.2-win\istio-1.29.2> kubectl apply -f samples/addons/jaeger.yaml
serviceaccount/grafana created
configmap/grafana created
service/grafana created
deployment.apps/grafana created
configmap/istio-grafana-dashboards created
configmap/istio-services-grafana-dashboards created
deployment.apps/jaeger created
service/tracing created
configmap/jaeger created
service/zipkin created
service/jaeger-collector created
PS C:\istio-1.29.2-win\istio-1.29.2>
```



```
PS C:\WINDOWS\system32> for ($i=1; $i -le 30; $i++) {
>>   try {
>>     $response = Invoke-WebRequest -Uri "http://localhost:8080/productpage" -UseBasicParsing
>>     Write-Host "Request $i : OK"
>>   } catch {
>>     Write-Host "Request $i : Failed"
>>   }
>>   Start-Sleep -Milliseconds 500
>> }
Request 1 : Failed
Request 2 : Failed
Request 3 : Failed
Request 4 : Failed
Request 5 : Failed
Request 6 : Failed
Request 7 : Failed
Request 8 : Failed
Request 9 : Failed
Request 10 : Failed
Request 11 : Failed
Request 12 : Failed
Request 13 : Failed
Request 14 : Failed
```

```
PS C:\WINDOWS\system32> kubectl port-forward svc/bookinfo-gateway-istio 8080:80
Forwarding from [::1]:8080 -> 80
Handling connection for 8080
Handling connection for 8080
```